BİL4010

Paralel programlama

15.Hafta

OpenMP

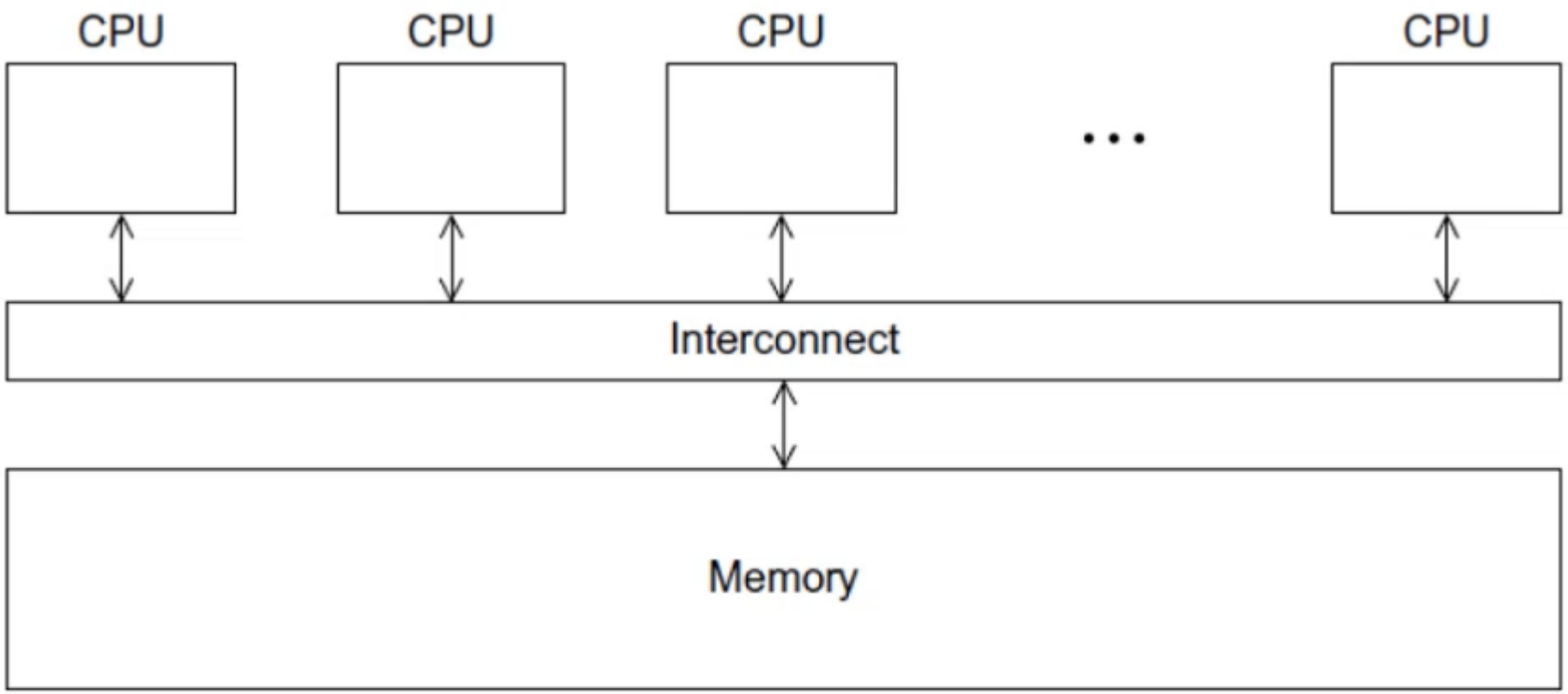
İçerik

- Hafıza Paylaşımlı Sistemler (Shared Memory Systems)
- OpenMP Nedir?
- #pragma kullanımı
- OpenMP örneği
- OpenMP programlarının derlenmesi ve çalıştırılması

Hafıza Paylaşımlı Sistemler

- Hafıza paylaşımlı sistemlerde, özerk işlemciler topluluğu, arabağlantı ağı (interconnection) üzerinden bir bellek sistemine bağlanmaktadır.
- Her bir işlemci tüm bellek konumuna erişebilir.
- Paylaşılan bellek sisteminde işlemciler genellikle paylaşılan veri yapılarına erişerek dolaylı olarak iletişim kurarlar.

Hafıza Paylaşımlı Sistemler



A Shared-memory System (Hafıza Paylaşımlı Sistem)

OpenMP Nedir?

- Hafıza paylaşımlı sistemlerde programlama için bir API
- **MP = multiprocessing**
- Her bir iş parçacığının (thread) veya işlemin (process) potansiyel olarak mevcut tüm belleğe erişebildiği sistemler için tasarlanmıştır.
- Sistem, ana belleğe erişimi olan çekirdek ya da işlemci (CPU) topluluğu olarak adlandırılabilir.
- Bu sunumda OpenMP kütüphanesinin kullanımı anlatılacaktır.

#pragma kullanımı

- Özel önişlemci (#) talimatları
- Genellikle temel C kodlarının parçası olmayan olaylara izin vermek için kod içerisinde ön işlemci komutları eklenir
- Derleyici #pragma'ları desteklemez ve onları görmezden gelir.

#pragma

OpenMP Programını Visual Studio ile Derlemek

1) Visual Studio C/C++ araçları ile kurulumu

2) Açılan Projede yapılması gereken ayarlamalar:

- Proje/Özellikler/C/C++/Dil→ Open MP Desteği: Evet (/openmp)
- Proje/Özellikler/C/C++/Önceden Derlenmiş Başlıklar→ Önceden Derlenmiş Üst Bilgi: Önceden Derlenmiş Üst Bilgiler Kullanılmıyor

#pragma örneği

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void Hello(void);  /* Thread function */

int main(int argc, char* argv[]) {
   /* Get number of threads from command line */
   int thread_count = strtol(argv[1], NULL, 10);

#  pragma omp parallel num_threads(thread_count)
   Hello();

   return 0;
}  /* main */

void Hello(void) {
   int my_rank = omp_get_thread_num();
   int thread_count = omp_get_num_threads();

   printf("Hello from thread %d of %d\n", my_rank, thread_count);

}  /* Hello */
```

OpenMP Kodunun çalıştırılması

- Başarılı bir şekilde derlenen x ismindeki programın çalıştırılması için x.exe dosyasının olduğu klasörün dizinine komut satırı üzerinden erişilerek: x.exe 4 komutu çalıştırılır
- Burada «4» parametresi bu programın kaç thread ile çalışacağını belirtmektedir.

Olası Program Çıktıları

İlk Çalıştırmada

```
Hello from thread 0 of 4
Hello from thread 1 of 4
Hello from thread 2 of 4
Hello from thread 3 of 4
```

İkinci Çalıştırmada

```
Hello from thread 1 of 4
Hello from thread 2 of 4
Hello from thread 0 of 4
Hello from thread 3 of 4
```

Diğer bir Çalıştırmada

```
Hello from thread 3 of 4
Hello from thread 1 of 4
Hello from thread 2 of 4
Hello from thread 0 of 4
```

Şeklinde çalışabilir. "Belirsizlik ilkesinden dolayı"

OpenMP #pragma kullanımı

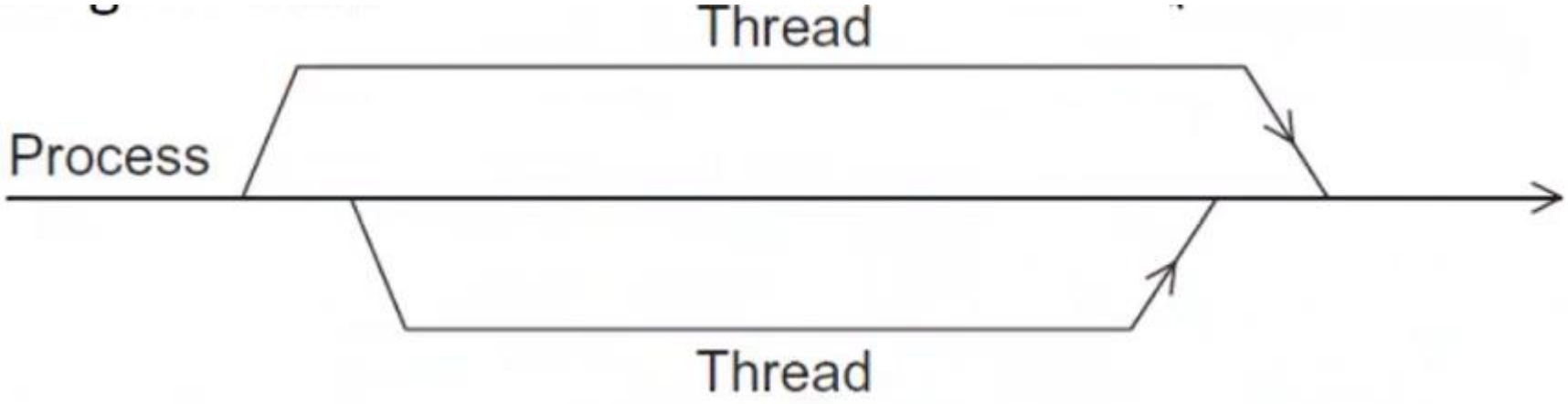
- Pragma derleyiciye ait özel bir «ön işlemci» komutudur.
- C dilinde compiler'a direktif vermek için kullanılır.
- Programlama esnasında genellikle global değişkenleri değiştirmede ve derleyiciyi yönlendirmede kullanılır.

```
# pragma omp parallel num_threads(thread_count)
```

- En basit paralel programlama komutudur.
- Bu ifadenin altındaki yapısal kod bloğunu çalıştıran iş parçacığı sayısı çalışma zamanında kullanıcı tarafından belirlenir ve işletim sistemine bildirir.

Threadlerin Dallanması

- Threadlerin dallanması ve kontrolü OpenMP tarafından gerçekleştirilir.
- Çalışma zamanında 1 işlem yaratılır ve paralel olarak istenilen miktarda iş parçacığı bu işlem üzerinden üretilir.
- İşlemini tamamlayan her bir iş parçacığı sonlanarak bağlı olduğu işleme geri katılır



strtol Fonksiyonu

- strtol fonksiyonu girilen string bir ifadeyi long int türüne çevirmek için kullanılır.
- strtol fonksiyonu kullanmak için <stdlib.h> kütüphanesinin import edilmesi gerekmektedir.
- Fonksiyona girdi olarak string bir ifade girilir ve çıkışta istenilen sayı tabanında long int türünde sayı üretir.
- 2. parametre NULL olarak kullanılabilir.

```
long strtol(
        const char*    number_p    /* in  */,
        char**         end_p       /* out */,
        int            base        /* in  */);
```

Rütbe ve İş Parçacığı sayısı belirlenmesi

- OpenMP programlarında her bir iş parçacığı bir rütbe (rank) alır:

```
int omp_get_thread_num(void);
```

- OpenMP programlarında toplam iş parçacığı sayısı aşağıdaki fonksiyon ile belirlenir:

```
int omp_get_num_threads(void);
```

```
int my_rank = omp_get_thread_num();
int thread_count = omp_get_num_threads();
```

OpenMP kütüphanesi

- OpenMP kütüphanesinin eklenmesi

```
#include <omp.h>
```

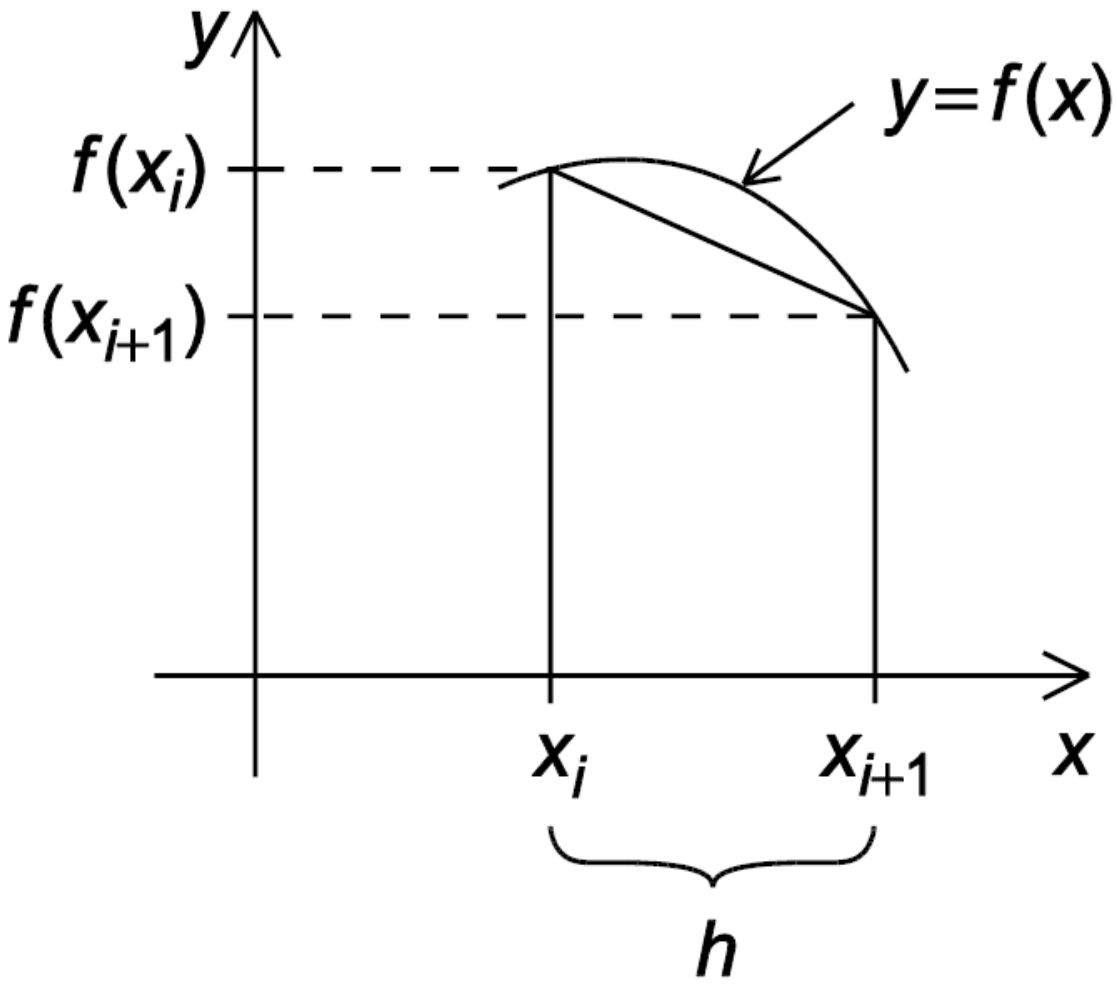
- Bazı derleyicilerde aşağıdaki şekilde de tanımlama yaparak hata kontrolü sağlanabilir.

```
#ifdef _OPENMP
#include <omp.h>
#endif
```

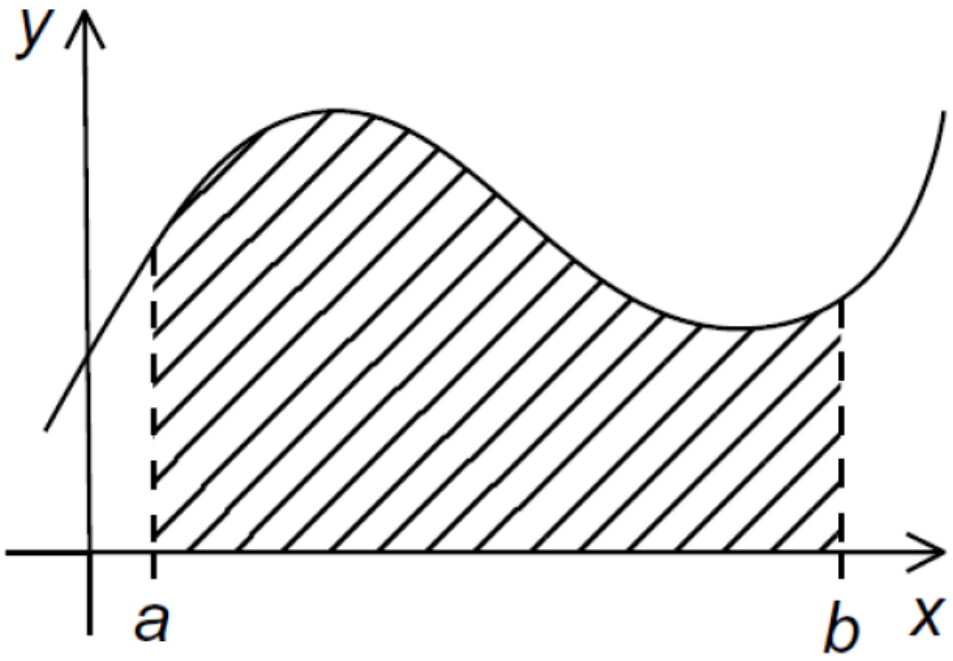
Hata Kontrolü

```
#ifdef _OPENMP
int my_rank=omp_get_thread_num( );
int thread_count=omp_get_num_threads( );
#else
İnt my_rank=0;
İnt thread_count=1;
#endif
```

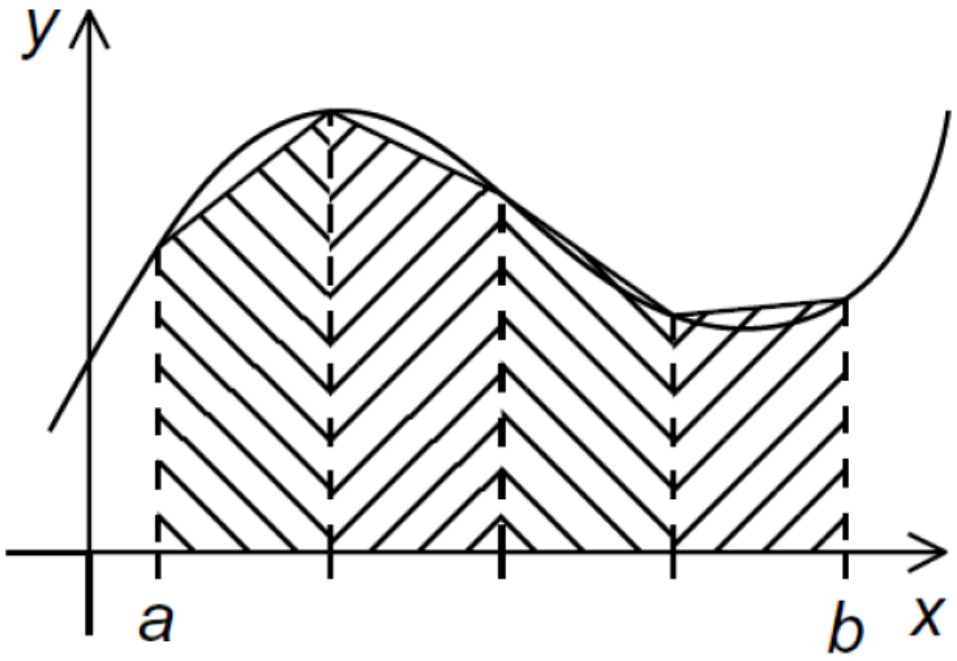
Bir Trapezoid için Alan Hesabı



Trapezoidal (Yamuk) Kuralı



(a)
Gerçek Alan



(b)
Trapezoid Kullanılarak Hesaplanan Yaklaşık Alan

Trapezoidal (Yamuk) Kuralı

$$\textit{Bir Trapezoidin Alanı} = \frac{h}{2}[f(x_i) + f(x_{i+1})]$$

Fonksiyonun altında kalan alanı n adet eşit parçaya böldüğümüzde her birinin yüksekliği

$$h = \frac{b-a}{n}$$

$$x_0 = a,\ x_1 = a+h,\ x_2 = a+2h,\ \ldots,\ x_{n-1} = a+(n-1)h,\ x_n = b$$

x eksenini n adet eşit parçaya bölerek alanı hesapladığımızda:

$$\int_{X_0}^{X_n} f(x)dx = \frac{h}{2}\left[\left(f(x_0) + f(x_n)\right) + 2(f(x_1) + f(x_2) + \cdots + f(x_{n-1}))\right]$$

$$\textit{Toplam Trapezoid Alanları} = h[f(x_0)/2 + f(x_1) + f(x_2) + \cdots + f(x_{n-1}) + f(x_n)/2]$$

OpenMP Trapezoid

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void Trap(double a, double b, int n, double* global_result_p);

int main(int argc, char* argv[]) {
   double  global_result = 0.0;  /* Store result in global_result */
   double  a, b;                 /* Left and right endpoints      */
   int     n;                    /* Total number of trapezoids    */
   int     thread_count;

   thread_count = strtol(argv[1], NULL, 10);
   printf("Enter a, b, and n\n");
   scanf("%lf %lf %d", &a, &b, &n);
#  pragma omp parallel num_threads(thread_count)
   Trap(a, b, n, &global_result);

   printf("With n = %d trapezoids, our estimate\n", n);
   printf("of the integral from %f to %f = %.14e\n",
      a, b, global_result);
   return 0;
}  /* main */
```

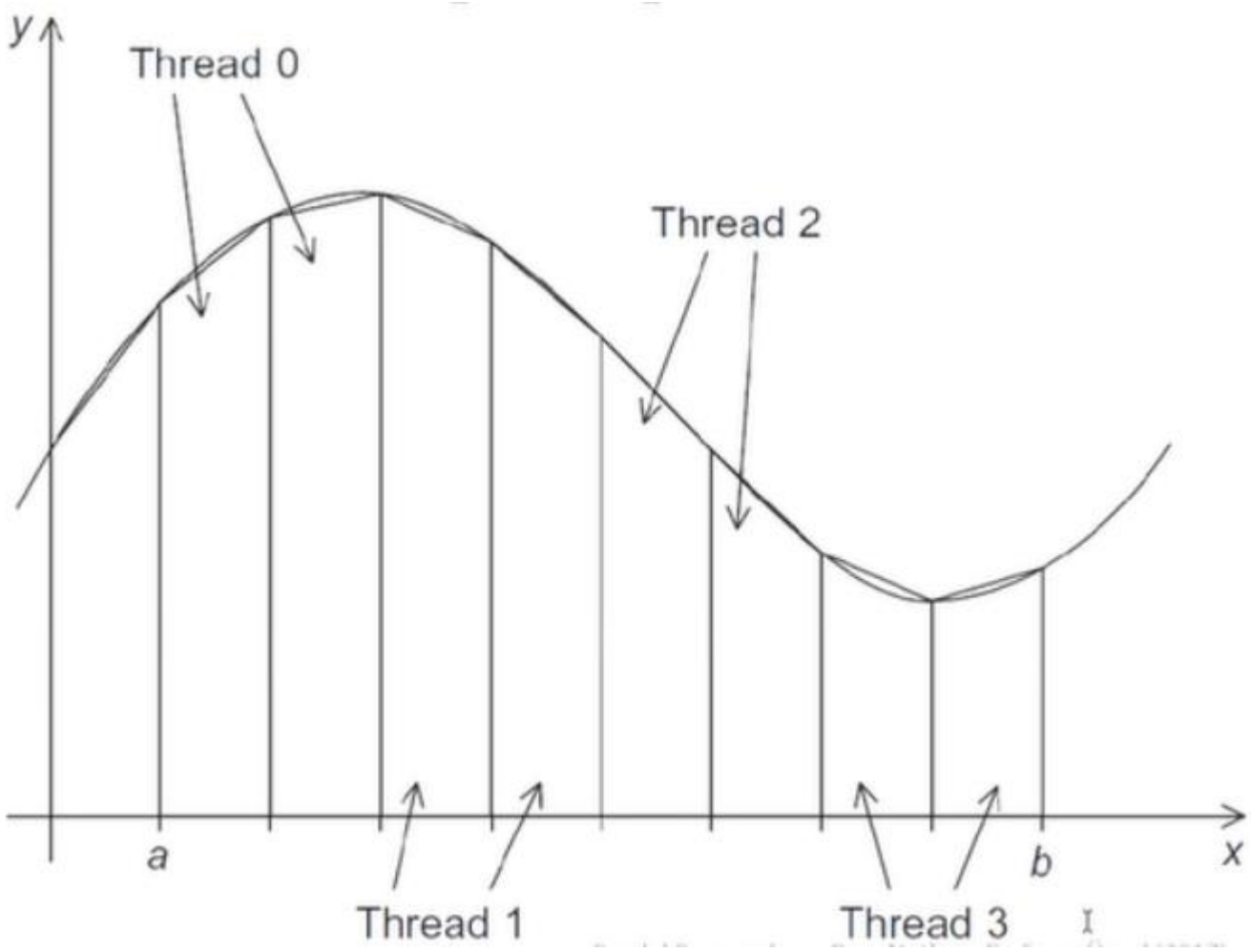
OpenMP Trapezoid

```
void Trap(double a, double b, int n, double* global_result_p) {
   double  h, x, my_result;
   double  local_a, local_b;
   int  i, local_n;
   int my_rank = omp_get_thread_num();
   int thread_count = omp_get_num_threads();

   h = (b-a)/n;
   local_n = n/thread_count;
   local_a = a + my_rank*local_n*h;
   local_b = local_a + local_n*h;
   my_result = (f(local_a) + f(local_b))/2.0;
   for (i = 1; i <= local_n-1; i++) {
     x = local_a + i*h;
     my_result += f(x);
   }
   my_result = my_result*h;

#  pragma omp critical
   *global_result_p += my_result;
}  /* Trap */
```

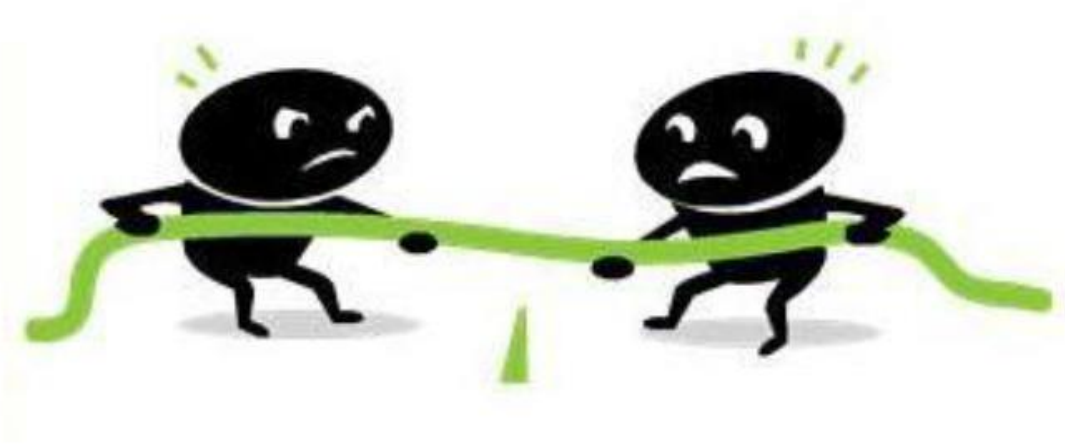
Yamuklara iş parçacıklarının atanması



Öngörülemeyen sonuçlar

- İki veya daha fazla iş parçacığı aynı anda çalıştırmaya kalktığında öngörülemeyen sonuçlar ortaya çıkabilir:

```
global_result += my_result;
```



Mutual Exclusion (Karşılıklı Dışlama)

- Bir program içerisinde ortak olarak paylaşılan alanlara erişim yapılan kod bölümlerine kritik bölge (critical section) denilir.
- Yarış durumlarından sakınmak için karşılıklı dışlama (mutual exclusion) yöntemi kullanılır.
- Bir iş parçacığı ortak değişken üzerinde işlem yapıyorsa başka bir iş parçacığı bu değişken üzerinde işlem yapamaz ve dışlanır.

```
# pragma omp critical
  global_result += my_result ;
```

- Yalnızca bir iş parçacığı yukarıdaki bloğu bir defada çalıştırabilir.

Değişkenlerin kapsamı

- Seri programlamada bir değişkenin kapsamı bir program içindeki değişkenin kullanılabileceği kısımlardan oluşur.
- OpenMP'de bir değişkenin kapsamı paralel bir blokta değişkene erişebilen iş parçacıkları grubu ile ifade edilir.
  - Shared (Genel) Kapsam: Gruptaki tüm iş parçacıkları tarafından erişilebilen değişkenlerdir.
  - Private (Özel) Kapsam: Yalnızca tek bir iş parçacığı tarafından erişilebilen değişkenlerdir.
- Not: Paralel bloktan önce tanımlanan değişkenlerin tamamı genel kapsamlıdır.

Teşekkürler.

Sorular...